TALLER FORMATIVO DE PROCESAMIENTO DISTRIBUIDO: PATRÓN MASTER-WORKER

Multiplicación Paralela de Matrices en Clúster con ZeroC Ice

FACULTAD DE INGENIERÍA, DISEÑO Y CIENCIAS APLICADAS INTELIGENTES

DEPARTAMENTO DE COMPUTACIÓN Y SISTEMAS

INFORMACIÓN DE LA ACTIVIDAD — IDENTIFICACIÓN

CÓDIGO:	09810 - TIC	**PERÍODO:**	202620 (Semestre 2026-2)
NRC / GRUPO:	12378 / 12377	**CARÁCTER:**	Práctica Formativa de Aprendizaje Activo
MODALIDAD:	Equipos de 2 o 3 Estudiantes	**DURACIÓN:**	3 Horas (En Sala de Cómputo / Trabajo Autónomo)
POLÍTICA IAG:	Nivel 3: Colaboración Asistida	**TEMA CENTRAL:**	Procesamiento Distribuido (MIMD / NORMA)
ESTUDIANTE 1:	_____	**CÓDIGO:**	_____
ESTUDIANTE 2:	_____	**CÓDIGO:**	_____
ESTUDIANTE 3:	_____	**CÓDIGO:**	_____

1 Contexto del Problema y Desafío de Ingeniería

Hasta este punto del curso de **Computación en Internet I**, hemos consolidado los fundamentos para comunicar dos componentes a través de la red:

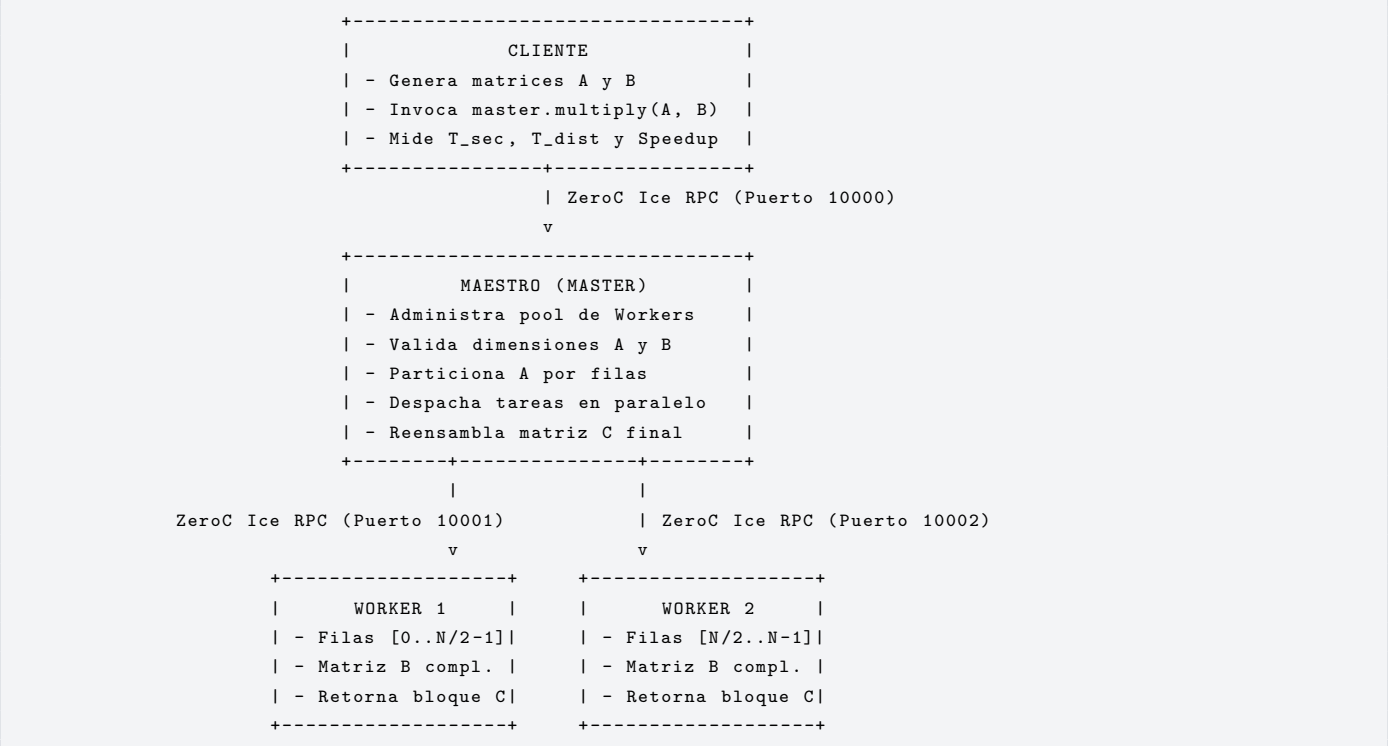
1. **Capa de Transporte:** Sockets crudos en Java (`TCP` y `UDP`) para transferencia de flujos de bytes y datagramas.
2. **Concurrencia en un Solo Nodo:** Manejo de hilos (`Thread`, `Runnable`), servidores multihilos y sincronización en memoria local.
3. **Middleware y RPC (ZeroC Ice):** Contratos formales en lenguaje Slice (`.ice`), tipado de red estricto, despacho asíncrono y proxies.

Sin embargo, **saber comunicar dos nodos por la red es solo la infraestructura base**. El desafío de ingeniería consiste en responder a la pregunta fundamental: **¿Cómo aprovechamos esa capacidad de comunicación para diseñar aplicaciones distribuidas capaces de resolver problemas computacionales pesados que una sola máquina no puede o tarda demasiado en procesar?**

En este taller implementaremos el patrón arquitectónico **Master-Worker** para resolver la **multiplicación de matrices masivas (**$C = A \times B$**)**, un algoritmo canónico con complejidad cúbica $\mathcal{O}(n^3)$. El sistema operará bajo la taxonomía de Flynn en la categoría **MIMD** (Multiple Instruction, Multiple Data) y el modelo de almacenamiento **NORMA** (*No-Remote Memory Access / Shared-Nothing*), donde ningún nodo comparte memoria física y toda coordinación se realiza mediante llamadas RPC sobre **ZeroC Ice**.

2 Topología del Sistema: Patrón Master-Worker

El sistema distribuido está compuesto por tres tipos de nodos independientes desplegados en la red:



- **Nodo Cliente (`client`):** Genera matrices de prueba pseudo-aleatorias, ejecuta una multiplicación monohilo local para establecer la línea base de tiempo (T_{sec}), solicita la multiplicación distribuida al Master (T_{dist}), verifica que los resultados numéricos sean idénticos celda por celda y calcula la **Aceleración (*Speedup* S)** y la **Eficiencia (E)**.
- **Nodo Maestro (`server / master`):** Escucha en el puerto `10000`. Mantiene un registro concurrente de Workers disponibles. Al recibir una petición, divide la matriz A en bloques contiguos de filas (grano grueso) y delega concurrentemente el cálculo a los Workers mediante `CompletableFuture`. Ensambla las filas recibidas y retorna la matriz C completa.
- **Nodo Trabajador (`worker`):** Escucha en un puerto configurable (ej. `10001`, `10002`, etc.). Al iniciar, se registra automáticamente ante el Master. Al recibir un bloque de filas de A y la matriz B completa, realiza el producto escalar local y retorna el bloque computado con la métrica de tiempo de cómputo.

3 Estructura del Proyecto Gradle Multimódulo

El código se organiza como un proyecto multimodular en **Gradle 8.6** gestionado por el plugin oficial de ZeroC (`com.zeroc.gradle.ice-build`):

```
taller-matrices-master-worker/
|-- settings.gradle                    # Orquestador de modulos incluidos
|-- build.gradle                       # Configuracion global de Java y dependencias Ice
|-- common/                            # Subproyecto del Contrato Slice (.ice)
|   |-- build.gradle                   # Plugin oficial ice-builder.slice
|   `-- src/main/slice/
|       `-- MatrixCompute.ice          # Definicion del contrato IDL
|-- server/                            # Subproyecto del Servidor Maestro (Master)
|   |-- build.gradle
|   `-- src/main/java/co/icesi/matrix/master/
|       |-- MasterMain.java            # Inicializacion de Communicator y Adapter
|       `-- MasterServiceI.java        # Servant: Particionamiento y despacho paralelo
|-- worker/                            # Subproyecto del Nodo Trabajador (Worker)
|   |-- build.gradle
|   `-- src/main/java/co/icesi/matrix/worker/
```

```
|       |-- WorkerMain.java                # Inicializacion y auto-registro en el Master
|       `-- WorkerServiceI.java           # Servant: Computo matricial intensivo
`-- client/                                # Subproyecto del Cliente
    |-- build.gradle
    `-- src/main/java/co/icesi/matrix/client/
        `-- ClientMain.java                # Generacion de matrices y calculo de Speedup
```

4 Contrato Slice (`MatrixCompute.ice`) y Configuración Gradle

4.1 Archivo 1: `common/src/main/slice/MatrixCompute.ice`

En ZeroC Ice, el contrato Slice define los tipos de datos que viajarán serializados por la red. La definición de una secuencia de secuencias (`sequence<DoubleSeq>`) se proyecta en Java nativamente como una matriz bidimensional `double[][]`:

```
module MatrixCompute {

    // Secuencias tipadas: se proyectan en Java como double[] y double[][]
    sequence<double> DoubleSeq;
    sequence<DoubleSeq> MatrixData;

    // Estructura que encapsula el resultado devuelto por un Worker
    struct BlockResult {
        int startRow;
        MatrixData computedRows;
        long computeTimeMs;
        string workerId;
    };

    // Excepcion para reportar inconsistencias en dimensiones o fallos
    exception MatrixException {
        string reason;
    };

    // Servicio que expone cada nodo Trabajador (Worker)
    interface WorkerService {
        BlockResult computeBlock(int startRow, MatrixData rowsA, MatrixData matrixB)
            throws MatrixException;

        idempotent string getWorkerInfo();
    };

    // Servicio central que expone el Maestro (Master)
    interface MasterService {
        MatrixData multiply(MatrixData matrixA, MatrixData matrixB)
            throws MatrixException;

        void registerWorker(string workerId, string endpoint);
        void unregisterWorker(string workerId);
        idempotent int getWorkerCount();
    };

};
```

4.2 Archivo 2: `settings.gradle` y `build.gradle` (Raíz)

```
// settings.gradle
rootProject.name = 'matrix-master-worker'
include 'common', 'worker', 'server', 'client'
```

```
// build.gradle (Raiz)
plugins {
    id 'com.zeroc.gradle.ice-builder.slice' version '1.5.0' apply false
}

allprojects {
    repositories {
        mavenCentral()
    }
}

subprojects {
    apply plugin: 'java'
    java {
        sourceCompatibility = JavaVersion.VERSION_17
        targetCompatibility = JavaVersion.VERSION_17
    }
    dependencies {
        implementation 'com.zeroc:ice:3.7.10'
    }
}
```

5 Implementación del Nodo Trabajador (Worker)

Cada Worker tiene como única responsabilidad recibir un subconjunto de filas de la matriz A y la matriz B completa, calcular las multiplicaciones escalares correspondientes en un arreglo local y retornar un objeto `BlockResult`.

5.1 Archivo 3: worker/src/main/java/co/icesi/matrix/worker/WorkerServiceI.java

```
package co.icesi.matrix.worker;

import MatrixCompute.BlockResult;
import MatrixCompute.MatrixException;
import MatrixCompute.WorkerService;
import com.zeroc.Ice.Current;

public class WorkerServiceI implements WorkerService {

    private final String workerId;

    public WorkerServiceI(String workerId) {
        this.workerId = workerId;
    }

    @Override
    public BlockResult computeBlock(int startRow, double[][] rowsA, double[][] matrixB, Current current)
            throws MatrixException {
        long startTime = System.currentTimeMillis();

        if (rowsA == null || matrixB == null || rowsA.length == 0 || matrixB.length == 0) {
            throw new MatrixException("Matrices nulas o vacias recibidas en worker " + workerId);
        }

        int numRowsA = rowsA.length;
        int numColsA = rowsA[0].length;
        int numRowsB = matrixB.length;
        int numColsB = matrixB[0].length;

        if (numColsA != numRowsB) {
```

```
            throw new MatrixException("Dimensiones incompatibles en worker: cols(A)=" + numColsA + " != rows(B)=" +
                 numRowsB);
        }

        System.out.printf("[%s] Computando bloque: fila inicio=%d (%d filas) contra B[%dx%d]%n",
                workerId, startRow, numRowsA, numRowsB, numColsB);

        // Multiplicacion matricial local para el bloque de filas asignado
        double[][] computedRows = new double[numRowsA][numColsB];
        for (int i = 0; i < numRowsA; i++) {
            for (int j = 0; j < numColsB; j++) {
                double sum = 0.0;
                for (int k = 0; k < numColsA; k++) {
                    sum += rowsA[i][k] * matrixB[k][j];
                }
                computedRows[i][j] = sum;
            }
        }

        long computeTime = System.currentTimeMillis() - startTime;
        System.out.printf("[%s] Bloque fila %d finalizado en %d ms%n", workerId, startRow, computeTime);

        return new BlockResult(startRow, computedRows, computeTime, workerId);
    }

    @Override
    public String getWorkerInfo(Current current) {
        return "WorkerID: " + workerId + " | Estado: ACTIVO | Cores: " + Runtime.getRuntime().availableProcessors()
             ;
    }
}
```

5.2 Archivo 4: worker/src/main/java/co/icesi/matrix/worker/WorkerMain.java

```
package co.icesi.matrix.worker;

import MatrixCompute.MasterServicePrx;
import com.zeroc.Ice.*;

public class WorkerMain {
    public static void main(String[] args) {
        int workerPort = (args.length >= 1) ? Integer.parseInt(args[0]) : 10001;
        String workerId = (args.length >= 2) ? args[1] : "Worker-" + workerPort;
        String masterHost = (args.length >= 3) ? args[2] : "localhost";
        int masterPort = (args.length >= 4) ? Integer.parseInt(args[3]) : 10000;

        try (Communicator communicator = Util.initialize(args)) {
            // 1. Crear el adaptador de objetos propio del Worker
            String endpoint = "tcp -h 0.0.0.0 -p " + workerPort;
            ObjectAdapter adapter = communicator.createObjectAdapterWithEndpoints("WorkerAdapter", endpoint);
            adapter.add(new WorkerServiceI(workerId), Util.stringToIdentity("WorkerService"));
            adapter.activate();

            // 2. Auto-registro dinamico ante el Master
            String masterEndpoint = String.format("MasterService:tcp -h %s -p %d", masterHost, masterPort);
            try {
                MasterServicePrx master = MasterServicePrx.checkedCast(communicator.stringToProxy(masterEndpoint));
                if (master != null) {
                    master.registerWorker(workerId, "tcp -h localhost -p " + workerPort);
                    System.out.printf("[%s] Registrado con exito en Master (%s)%n", workerId, masterEndpoint);
                }
```

```
            } catch (Exception e) {
                System.err.println("[WARN] No se pudo conectar al Master: " + e.getMessage());
            }

            System.out.printf("[%s] Escuchando en %s. Presione Ctrl+C para salir.%n", workerId, endpoint);
            communicator.waitForShutdown();
        }
    }
}
```

6 Implementación del Nodo Maestro (Master)

El Master es el orquestador del clúster. Al recibir la matriz A de tamaño $M \times K$ y B de $K \times N$, determina cuántos Workers están registrados, particiona A horizontalmente en bloques contiguos y lanza las llamadas RPC concurrentes hacia los workers usando `CompletableFuture.supplyAsync()`.

6.1 Archivo 5: server/src/main/java/co/icesi/matrix/master/MasterServiceI.java

```
package co.icesi.matrix.master;

import MatrixCompute.*;
import com.zeroc.Ice.*;
import java.util.*;
import java.util.concurrent.*;

public class MasterServiceI implements MasterService {
    private final Map<String, WorkerServicePrx> workers = new ConcurrentHashMap<>();

    @Override
    public double[][] multiply(double[][] matrixA, double[][] matrixB, Current current) throws MatrixException {
        long t0 = System.currentTimeMillis();
        int rowsA = matrixA.length;
        int colsA = matrixA[0].length;
        int rowsB = matrixB.length;
        int colsB = matrixB[0].length;

        if (colsA != rowsB) {
            throw new MatrixException("Dimensiones incompatibles: cols(A)=" + colsA + " != rows(B)=" + rowsB);
        }

        List<WorkerServicePrx> activeWorkers = new ArrayList<>(workers.values());
        if (activeWorkers.isEmpty()) {
            throw new MatrixException("No hay Workers registrados en el Master.");
        }

        int numWorkers = activeWorkers.size();
        int rowsPerWorker = (int) Math.ceil((double) rowsA / numWorkers);
        List<CompletableFuture<BlockResult>> futures = new ArrayList<>();

        int currentRow = 0;
        for (int w = 0; w < numWorkers && currentRow < rowsA; w++) {
            int startRow = currentRow;
            int count = Math.min(rowsPerWorker, rowsA - startRow);

            // Extraer bloque de filas de A
            double[][] sliceA = new double[count][colsA];
            for (int r = 0; r < count; r++) sliceA[r] = matrixA[startRow + r];

            WorkerServicePrx worker = activeWorkers.get(w);
```

```
            // Despacho asincrono concurrente hacia el Worker
            futures.add(CompletableFuture.supplyAsync(() -> {
                try {
                    return worker.computeBlock(startRow, sliceA, matrixB);
                } catch (MatrixException e) {
                    throw new RuntimeException(e);
                }
            }));
            currentRow += count;
        }

        // Reensamblado de la matriz C final a partir de los bloques recolectados
        double[][] resultC = new double[rowsA][colsB];
        for (CompletableFuture<BlockResult> future : futures) {
            BlockResult block = future.join();
            for (int i = 0; i < block.computedRows.length; i++) {
                resultC[block.startRow + i] = block.computedRows[i];
            }
        }

        System.out.printf("[MASTER] Multiplicacion completada en %d ms usando %d workers.%n",
                (System.currentTimeMillis() - t0), numWorkers);
        return resultC;
    }

    @Override
    public void registerWorker(String workerId, String endpoint, Current current) {
        try {
            Communicator comm = current.adapter.getCommunicator();
            WorkerServicePrx prx = WorkerServicePrx.checkedCast(comm.stringToProxy("WorkerService:" + endpoint));
            if (prx != null) {
                workers.put(workerId, prx);
                System.out.printf("[MASTER] [+] Worker registrado: %s (%s) | Total: %d%n", workerId, endpoint,
                    workers.size());
            }
        } catch (Exception e) {
            System.err.println("[MASTER] Error registrando worker " + workerId + ": " + e.getMessage());
        }
    }

    @Override
    public void unregisterWorker(String workerId, Current current) {
        workers.remove(workerId);
        System.out.printf("[MASTER] [-] Worker retirado: %s | Total: %d%n", workerId, workers.size());
    }

    @Override
    public int getWorkerCount(Current current) { return workers.size(); }
}
```

6.2 Archivo 6: server/src/main/java/co/icesi/matrix/master/MasterMain.java

```
package co.icesi.matrix.master;

import com.zeroc.Ice.*;

public class MasterMain {
    public static void main(String[] args) {
        int port = (args.length >= 1) ? Integer.parseInt(args[0]) : 10000;
        try (Communicator communicator = Util.initialize(args)) {
```

```
            ObjectAdapter adapter = communicator.createObjectAdapterWithEndpoints("MasterAdapter", "tcp -h 0.0.0.0
                -p " + port);
            adapter.add(new MasterServiceI(), Util.stringToIdentity("MasterService"));
            adapter.activate();
            System.out.printf("[MASTER] Escuchando en puerto %d. Esperando peticiones...%n", port);
            communicator.waitForShutdown();
        }
    }
}
```

7 Implementación del Nodo Cliente y Benchmark de Speedup

El cliente se conecta al Master, efectúa una multiplicación de verificación funcional (4x4) y posteriormente ejecuta un benchmark masivo comparando el tiempo secuencial monohilo (T_{sec}) contra el clúster distribuido (T_{dist}).

7.1 Archivo 7: client/src/main/java/co/icesi/matrix/client/ClientMain.java

```
package co.icesi.matrix.client;

import MatrixCompute.MasterServicePrx;
import com.zeroc.Ice.*;
import java.util.Random;

public class ClientMain {
    public static void main(String[] args) {
        String host = (args.length >= 1) ? args[0] : "localhost";
        int port = (args.length >= 2) ? Integer.parseInt(args[1]) : 10000;
        int size = (args.length >= 3) ? Integer.parseInt(args[2]) : 500;

        try (Communicator communicator = Util.initialize(args)) {
            MasterServicePrx master = MasterServicePrx.checkedCast(
                communicator.stringToProxy("MasterService:tcp -h " + host + " -p " + port));

            int p = master.getWorkerCount();
            System.out.printf("Conectado al Master. Workers disponibles en el cluster: %d%n", p);

            // Generar matrices pseudo-aleatorias
            Random rand = new Random(42);
            double[][] A = new double[size][size];
            double[][] B = new double[size][size];
            for (int i = 0; i < size; i++) {
                for (int j = 0; j < size; j++) {
                    A[i][j] = rand.nextDouble() * 10.0;
                    B[i][j] = rand.nextDouble() * 10.0;
                }
            }

            // 1. Multiplicacion Monohilo Local
            long t0Sec = System.currentTimeMillis();
            double[][] C_local = new double[size][size];
            for (int i = 0; i < size; i++) {
                for (int j = 0; j < size; j++) {
                    double s = 0.0;
                    for (int k = 0; k < size; k++) s += A[i][k] * B[k][j];
                    C_local[i][j] = s;
                }
            }
            long tSec = System.currentTimeMillis() - t0Sec;

            // 2. Multiplicacion Distribuida en Cluster
```

```
            long t0Dist = System.currentTimeMillis();
            double[][] C_dist = master.multiply(A, B);
            long tDist = System.currentTimeMillis() - t0Dist;

            // 3. Metricas de Rendimiento
            double speedup = (double) tSec / (double) Math.max(1, tDist);
            double efficiency = (speedup / p) * 100.0;

            System.out.println("================================================");
            System.out.println("            RESULTADOS DEL BENCHMARK");
            System.out.println("================================================");
            System.out.printf("Dimension: %dx%d | Workers: %d%n", size, size, p);
            System.out.printf("Tiempo Secuencial (T_sec):   %d ms%n", tSec);
            System.out.printf("Tiempo Distribuido (T_dist): %d ms%n", tDist);
            System.out.printf("Aceleracion (Speedup S):     %.2fx%n", speedup);
            System.out.printf("Eficiencia Paralela (E):     %.1f%%%n", efficiency);
            System.out.println("================================================");
        }
    }
}
```

8 Protocolo de Ejecución Experimental en el Laboratorio

Abra 4 terminales independientes en su máquina o conéctese vía SSH a las estaciones de la Sala 205M:

1. **Terminal 1 (Iniciar Maestro):**

```
./gradlew :server:runMaster
```

2. **Terminal 2 (Iniciar Worker 1 en puerto 10001):**

```
./gradlew :worker:runWorker --args="10001 Worker-1 localhost 10000"
```

3. **Terminal 3 (Iniciar Worker 2 en puerto 10002):**

```
./gradlew :worker:runWorker --args="10002 Worker-2 localhost 10000"
```

4. **Terminal 4 (Lanzar Cliente con Matrices de 600x600):**

```
./gradlew :client:runClient --args="localhost 10000 600"
```

9 Cuestionario de Análisis Conceptual y Discusión Técnica

Responda con precisión técnica las siguientes preguntas basándose en las mediciones obtenidas:

1. **El Umbral de Amortización de Red:** Si ejecuta el benchmark con matrices pequeñas de 20×20, observará que el tiempo distribuido T_{dist} es sustancialmente mayor que el tiempo secuencial T_{sec} ($S < 1$). Explique técnicamente por qué ocurre este fenómeno analizando los costos de serialización de bytes en Slice, buffers de red y latencia TCP frente al cómputo aritmético.
2. **Ley de Amdahl y Eficiencia:** Al pasar de 1 a 2 workers y luego de 2 a 4 workers, ¿la aceleración S se duplica perfectamente? ¿Qué partes del proceso corresponden a la fracción estrictamente secuencial $(1 - f)$ que limita el crecimiento lineal?
3. **Patrón Shared-Nothing (NORMA):** En la memoria de un solo computador (UMA), todos los hilos leen la matriz B sin necesidad de duplicarla. En nuestra arquitectura distribuida sobre red, ¿cuántas copias de la matriz B existen simultáneamente en memoria? ¿Qué impacto tiene esto en el consumo de ancho de banda y qué optimización propondría?

4. **Tolerancia a Fallos:** ¿Qué ocurriría si uno de los workers se desconecta abruptamente en medio del cálculo de un bloque? ¿Cómo podría el Master detectar el fallo y reasignar dicho bloque a otro worker superviviente?

10 Lista de Autoverificación Formativa (Checklist)

☐ **Hito 1:** El contrato `MatrixCompute.ice` compila satisfactoriamente con `./gradlew :common:build`.

☐ **Hito 2:** El Master inicia en el puerto `10000` y registra dinámicamente múltiples Workers.

☐ **Hito 3:** Cada Worker calcula su bloque de filas asignado y retorna el tiempo de procesamiento local.

☐ **Hito 4:** El Master ensambla la matriz C completa sin corromper el orden de las filas.

☐ **Hito 5:** El Cliente valida que el resultado distribuido coincida numéricamente con el cálculo monohilo ($\Delta < 10^{-5}$).

☐ **Hito 6:** El benchmark evidencia una aceleración real ($S > 1$) para matrices de dimensión masiva ($N \geq 500$).